

Skill Week 6

Session 11

Task 1

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.



```
GNU nano 8.7.1 session11_history.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

/* Store a command in history */
void store_command(const char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        /* Remove the oldest command */
        for (int i = 1; i < MAX_HISTORY; i++)
        {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

/* Display command history */
void display_history()
{
    if (history_count == 0)
    {
        printf("History is empty.\n");
        return;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQQ... × + ▾
GNU nano 8.7.1 session11_history.c *

    for (int i = 0; i < history_count; i++)
    {
        printf("%d  %s\n", i + 1, history[i]);
    }
}

/* Retrieve a history entry */
void retrieve_command(int number)
{
    if (number < 1 || number > history_count)
    {
        printf("Invalid history number.\n");
        return;
    }

    printf("Command %d: %s\n", number, history[number - 1]);
}

int main()
{
    char command[MAX_COMMAND];

    printf("Simple Command History\n");
    printf("Maximum history size: %d\n", MAX_HISTORY);
    printf("Commands: history, get N, exit\n\n");

    while (1)
    {
        printf("history-shell$ ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```

    if (strlen(command) == 0)
        continue;

    if (strcmp(command, "exit") == 0)
    {
        break;
    }
    else if (strcmp(command, "history") == 0)
    {
        display_history();
    }
    else if (strncmp(command, "get ", 4) == 0)
    {
        int number = atoi(command + 4);
        retrieve_command(number);
    }
    else
    {
        store_command(command);
        printf("Command stored.\n");
    }
}

printf("\nFinal command history:\n");
display_history();

return 0;
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```
jella_pragathi@LAPTOP-RQQGT555:~$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ nano session11_history.c
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ gcc -Wall -Wextra -g session11_history.c -o session11_history
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session11_history
Simple Command History
Maximum history size: 10
Commands: history, get N, exit

history-shell$ ls
Command stored.
history-shell$ pwd
Command stored.
history-shell$ history
1  ls
2  pwd
history-shell$ get 1
Command 1: ls
history-shell$ get 2
Command 2: pwd
history-shell$ history
1  ls
2  pwd
history-shell$ exit

Final command history:
1  ls
2  pwd
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ #define MAX_HISTORY 10
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ one
Command 'one' not found, did you mean:
  command 'sone' from snap sone (0.21.0)
  command 'ode' from deb plotutils (2.6-15build1)
  command 'mne' from deb python3-mne (1.9.0-2)
  command 'ne' from deb ne (3.3.4-3)
See 'snap info <snapname>' for additional versions.
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session11_history
Simple Command History
Maximum history size: 10
Commands: history, get N, exit
```

```
history-shell$ one
Command stored.
history-shell$ two
Command stored.
history-shell$ three
Command stored.
history-shell$ four
Command stored.
history-shell$ five
Command stored.
history-shell$ history
1 one
2 two
3 three
4 four
5 five
history-shell$ exit
```

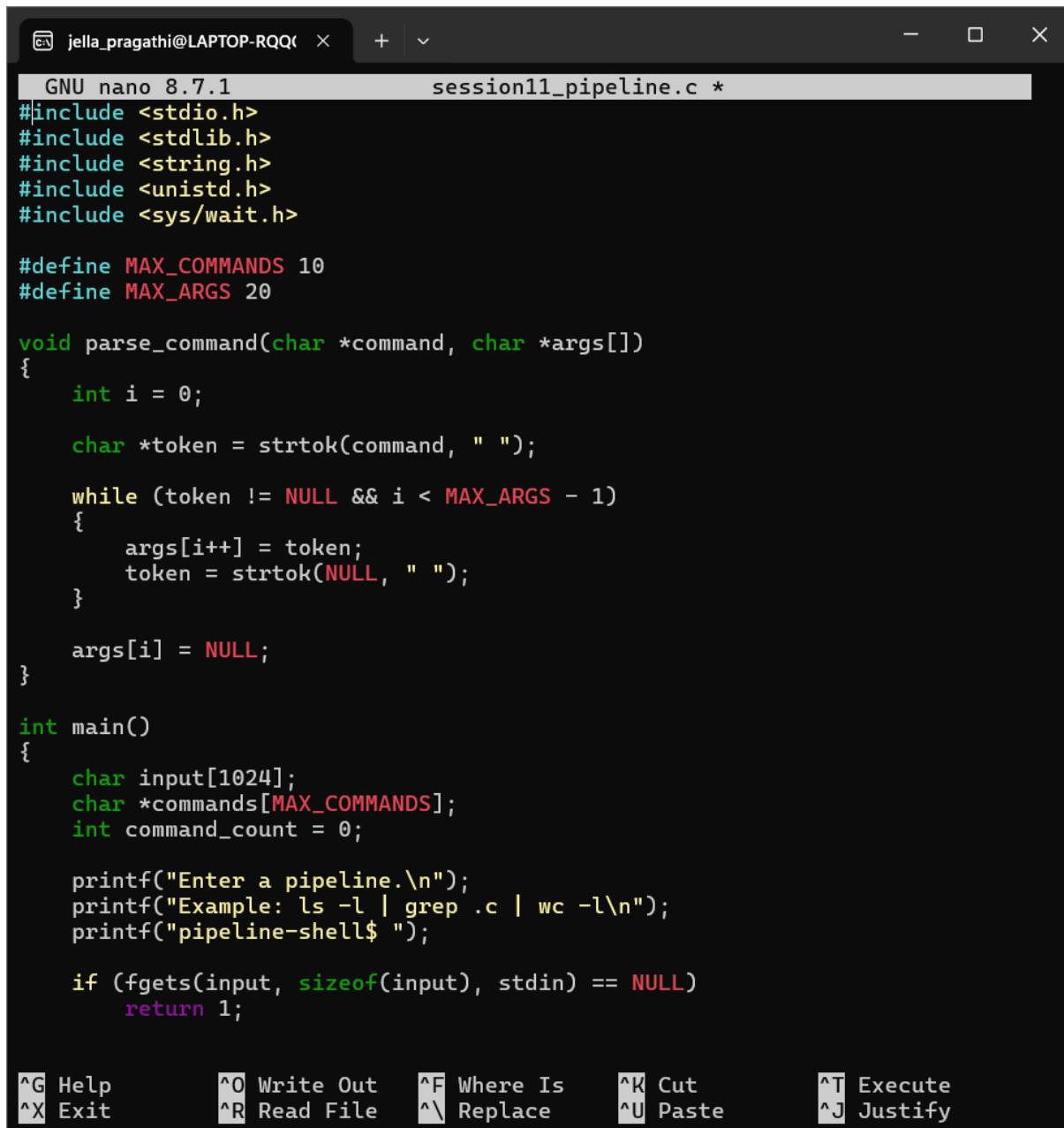
Final command history:

```
1 one
2 two
3 three
4 four
5 five
```

```
jella_pragathi@LAPTOP-RQQT555:~/OSSP$
```

Task 2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.



```
GNU nano 8.7.1 session11_pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20

void parse_command(char *command, char *args[])
{
    int i = 0;

    char *token = strtok(command, " ");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;
}

int main()
{
    char input[1024];
    char *commands[MAX_COMMANDS];
    int command_count = 0;

    printf("Enter a pipeline.\n");
    printf("Example: ls -l | grep .c | wc -l\n");
    printf("pipeline-shell$ ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```

    return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Split input using | */
    char *token = strtok(input, "|");

    while (token != NULL && command_count < MAX_COMMANDS)
    {
        commands[command_count++] = token;
        token = strtok(NULL, "|");
    }

    if (command_count == 0)
    {
        printf("No commands entered.\n");
        return 1;
    }

    printf("\nPipeline contains %d command(s):\n", command_count);

    for (int i = 0; i < command_count; i++)
    {
        printf("Command %d: %s\n", i + 1, commands[i]);
    }

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    /* Create pipes */
    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)

```

[^]G
[^]X

Help
 Exit

[^]O
[^]R

Write Out
 Read File

[^]F
[^]\

Where Is
 Replace

[^]K
[^]U

Cut
 Paste

[^]T
[^]J

Execute
 Justify

```
{
    perror("pipe");
    return 1;
}

/* Create processes */
for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork");
        return 1;
    }

    if (pids[i] == 0)
    {
        char *args[MAX_ARGS];

        parse_command(commands[i], args);

        /* Input from previous command */
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        /* Output to next command */
        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        /* Close pipe descriptors */
        for (int j = 0; j < command_count - 1; j++)
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```

        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    /* Execute command */
    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

/* Parent closes all pipes */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Wait for all child processes */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nPipeline execution completed.\n");

return 0;
}

```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

```

jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ nano session11_pipeline.c
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ gcc -Wall -Wextra -g session11_pipeline.c -o session11_pipeline
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session11_pipeline
Enter a pipeline.
Example: ls -l | grep .c | wc -l
pipeline-shell$ ls -l | grep .c

Pipeline contains 2 command(s):
Command 1: ls -l
Command 2: grep .c
drwxr-xr-x 2 jella_pragathi jella_pragathi 4096 Aug  1 08:01 docs
drwxr-xr-x 2 jella_pragathi jella_pragathi 4096 Aug  1 08:01 include
-rw-r--r-- 1 jella_pragathi jella_pragathi 1181 Aug 31 03:46 pipeline.c
-rwxr-xr-x 1 jella_pragathi jella_pragathi 19280 Aug 31 03:41 producer_consumer
-rw-r--r-- 1 jella_pragathi jella_pragathi 1865 Aug 31 03:41 producer_consumer.c
drwxr-xr-x 2 jella_pragathi jella_pragathi 4096 Aug  1 08:02 scripts
-rw-r--r-- 1 jella_pragathi jella_pragathi 1970 Sep 16 08:18 session10_export.c
-rw-r--r-- 1 jella_pragathi jella_pragathi  963 Sep 16 08:11 session10_pwd_exit.c
-rw-r--r-- 1 jella_pragathi jella_pragathi 2099 Sep 17 05:57 session11_history.c
-rw-r--r-- 1 jella_pragathi jella_pragathi 2792 Sep 17 06:04 session11_pipeline.c
-rw-r--r-- 1 jella_pragathi jella_pragathi 1507 Aug 23 16:14 session4_wait.c
-rwxr-xr-x 1 jella_pragathi jella_pragathi 20344 Sep 16 07:00 session9_cd
-rw-r--r-- 1 jella_pragathi jella_pragathi 1913 Sep 16 07:00 session9_cd.c
-rwxr-xr-x 1 jella_pragathi jella_pragathi 21184 Sep 16 07:07 session9_dispatch
-rw-r--r-- 1 jella_pragathi jella_pragathi 2359 Sep 16 07:07 session9_dispatch.c
drwxr-xr-x 2 jella_pragathi jella_pragathi 4096 Aug 17 05:05 src
-rwxr-xr-x 1 jella_pragathi jella_pragathi 16224 Aug 26 03:26 task1_escape
-rw-r--r-- 1 jella_pragathi jella_pragathi  572 Aug 26 03:25 task1_escape.c
-rwxr-xr-x 1 jella_pragathi jella_pragathi 16296 Aug 26 03:30 task2_child
-rw-r--r-- 1 jella_pragathi jella_pragathi  807 Aug 26 03:29 task2_child.c
-rw-r--r-- 1 jella_pragathi jella_pragathi  780 Aug 23 16:41 zombie.c

Pipeline execution completed.

```

```
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session11_pipeline
```

```
Enter a pipeline.
```

```
Example: ls -l | grep .c | wc -l
```

```
pipeline-shell$ ls -l | grep .c | wc -l
```

```
Pipeline contains 3 command(s):
```

```
Command 1: ls -l
```

```
Command 2: grep .c
```

```
Command 3: wc -l
```

```
21
```

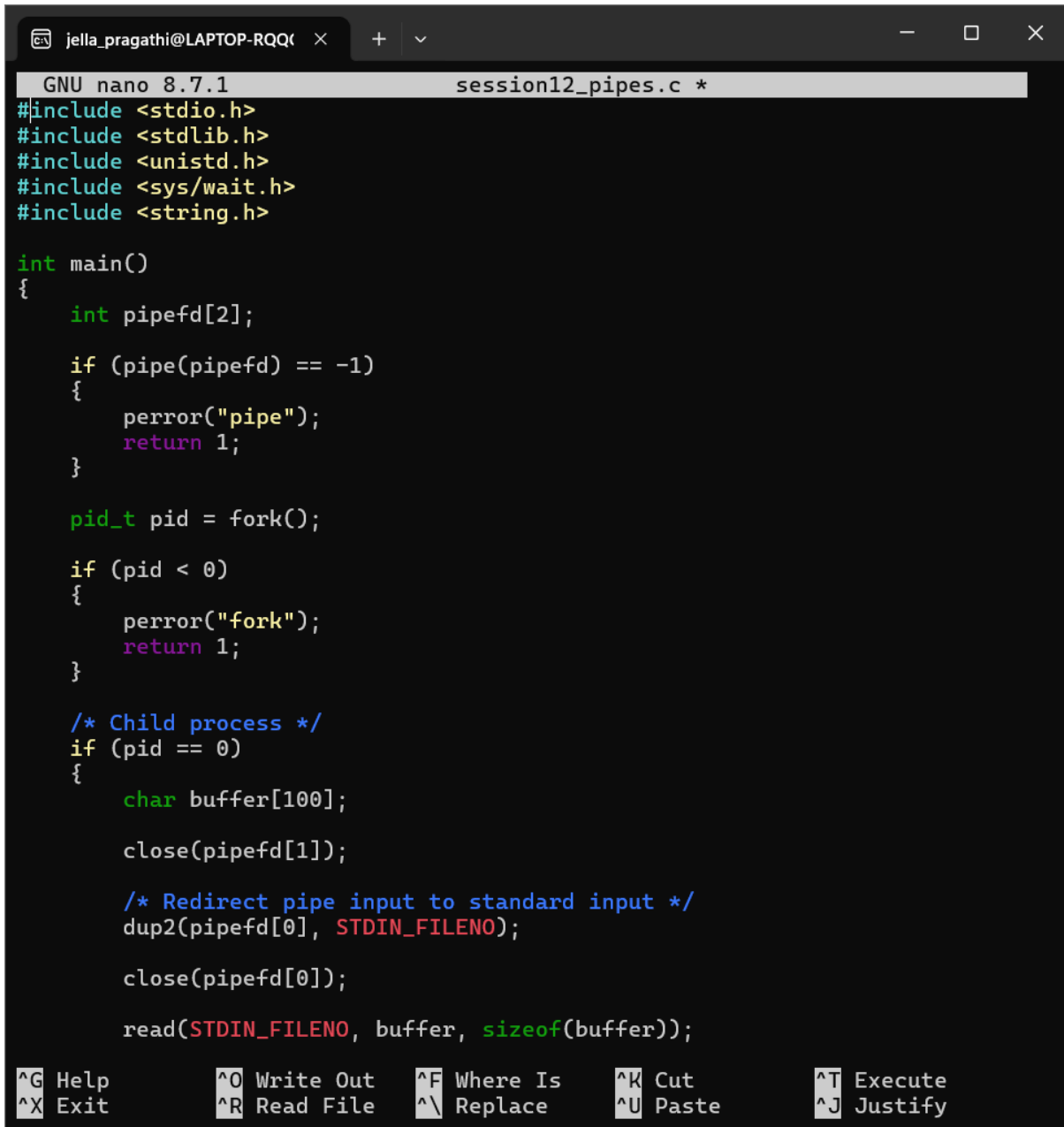
```
Pipeline execution completed.
```

```
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ |
```

skill Session 12

Task 1

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.



```
GNU nano 8.7.1 session12_pipes.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    /* Child process */
    if (pid == 0)
    {
        char buffer[100];

        close(pipefd[1]);

        /* Redirect pipe input to standard input */
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        read(STDIN_FILENO, buffer, sizeof(buffer));
    }
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

```

        printf("Child received: %s\n", buffer);

        exit(0);
    }

    /* Parent process */
    else
    {
        char message[] = "Hello from parent process!";

        close(pipefd[0]);

        /* Redirect standard output to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        printf("%s", message);
        fflush(stdout);

        waitpid(pid, NULL, 0);
    }

    return 0;
}
|

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

Pipeline execution completed.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ ./session11_pipeline
Enter a pipeline.
Example: ls -l | grep .c | wc -l
pipeline-shell$ ls -l | grep .c | wc -l

Pipeline contains 3 command(s):
Command 1: ls -l
Command 2: grep .c
Command 3: wc -l
21

Pipeline execution completed.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ cd ~/OSSP
nano session12_pipes.c
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ gcc -Wall -Wextra -g session12_pipes.
c -o session12_pipes
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ ./session12_pipes
Child received: Hello from parent process!@
jella_pragathi@LAPTOP-RQQT555:~/OSSP$

```

Task 2

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.



```
GNU nano 8.7.1 session12_multi_pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 1024

void parse_command(char *command, char *args[])
{
    int i = 0;

    char *token = strtok(command, " ");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];
    char *commands[MAX_COMMANDS];

    int command_count = 0;

    printf("Multi-Pipe Shell\n");
    printf("Example: ls -l | grep .c | sort | wc -l\n");
    printf("multi-pipe$ ");

    if (fgets(input, sizeof(input), stdin) == NULL)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```
char *token = strtok(input, "|");

while (token != NULL && command_count < MAX_COMMANDS)
{
    commands[command_count++] = token;
    token = strtok(NULL, "|");
}

if (command_count == 0)
{
    printf("No commands entered.\n");
    return 1;
}

printf("\nNumber of commands: %d\n", command_count);

for (int i = 0; i < command_count; i++)
{
    printf("Command %d: %s\n", i + 1, commands[i]);
}

/* Create pipe array */
int pipes[MAX_COMMANDS - 1][2];

/* Create required pipes */
for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");
        return 1;
    }
}

pid_t pids[MAX_COMMANDS];

/* Create one process for each command */
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
/* Create one process for each command */
for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork");
        return 1;
    }

    if (pids[i] == 0)
    {
        char *args[MAX_ARGS];

        parse_command(commands[i], args);

        /* Not first command: read from previous pipe */
        if (i > 0)
        {
            if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
            {
                perror("dup2 input");
                exit(1);
            }
        }

        /* Not last command: write to next pipe */
        if (i < command_count - 1)
        {
            if (dup2(pipes[i][1], STDOUT_FILENO) == -1)
            {
                perror("dup2 output");
                exit(1);
            }
        }
    }
}
```

```

        /* Close all pipe descriptors in child */
        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        /* Execute command */
        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }
}

/* Parent closes all pipe descriptors */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Wait for all child processes */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll pipeline processes completed.\n");

return 0;
}
|

```

^{^G} Help
^{^X} Exit

^{^O} Write Out
^{^R} Read File

^{^F} Where Is
^{^\} Replace

^{^K} Cut
^{^U} Paste

^{^T} Execute
^{^J} Justify

```
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ cd ~/OSSP
nano session12_multi_pipe.c
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ gcc -Wall -Wextra -g session12_multi_
pipe.c -o session12_multi_pipe
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ ./session12_multi_pipe
Multi-Pipe Shell
Example: ls -l | grep .c | sort | wc -l
multi-pipe$ ls -l | grep .c | wc -l

Number of commands: 3
Command 1: ls -l
Command 2:  grep .c
Command 3:  wc -l
23

All pipeline processes completed.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ ./session12_multi_pipe
Multi-Pipe Shell
Example: ls -l | grep .c | sort | wc -l
multi-pipe$ ls -l | grep .c | sort | wc -l

Number of commands: 4
Command 1: ls -l
Command 2:  grep .c
Command 3:  sort
Command 4:  wc -l
23

All pipeline processes completed.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ |
```